LAPORAN PRAKTIKUM

POSTTEST (2)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Antung Hissyam (2509106092)

Kelas (C1 ‘25)

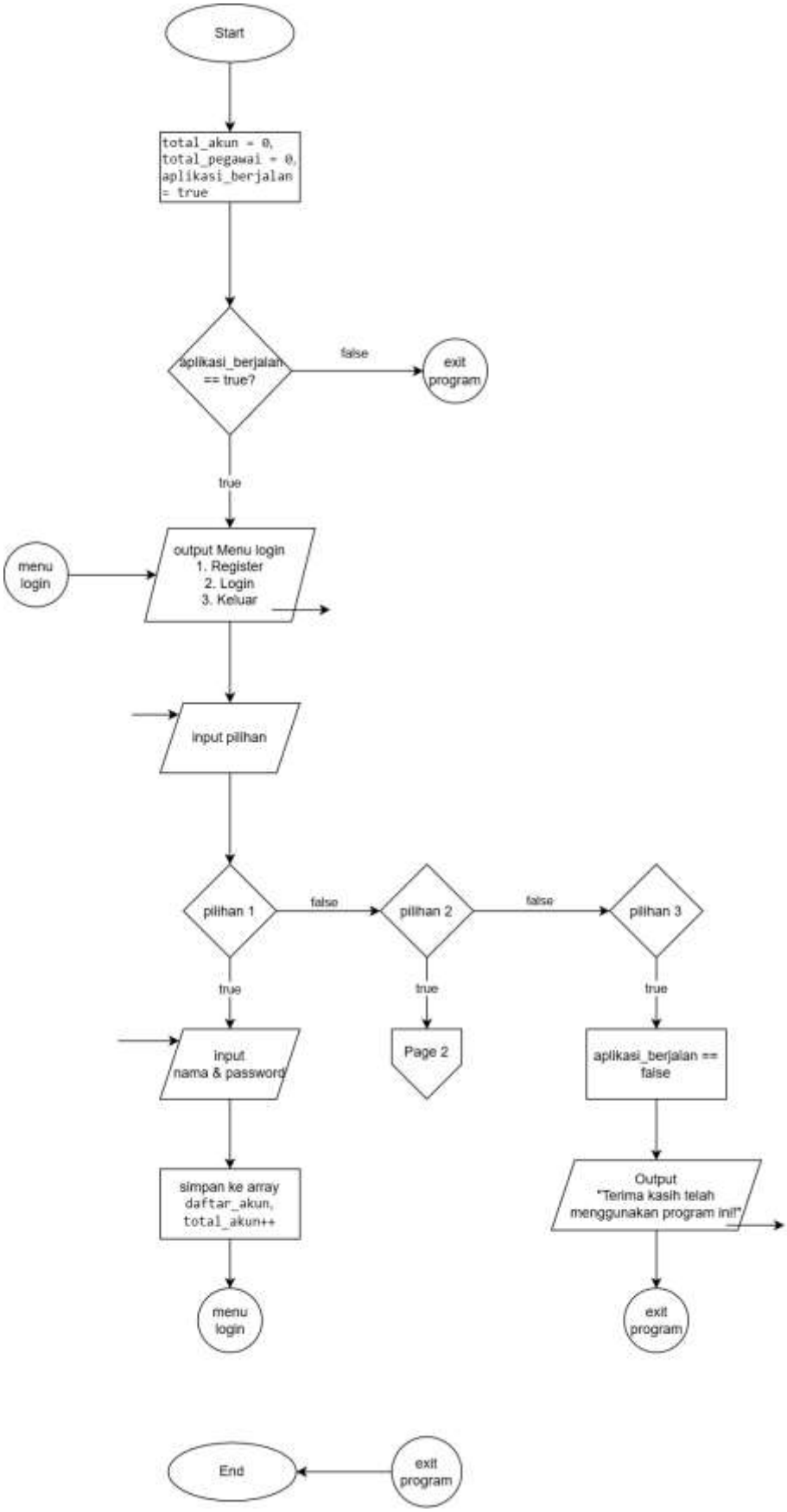
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

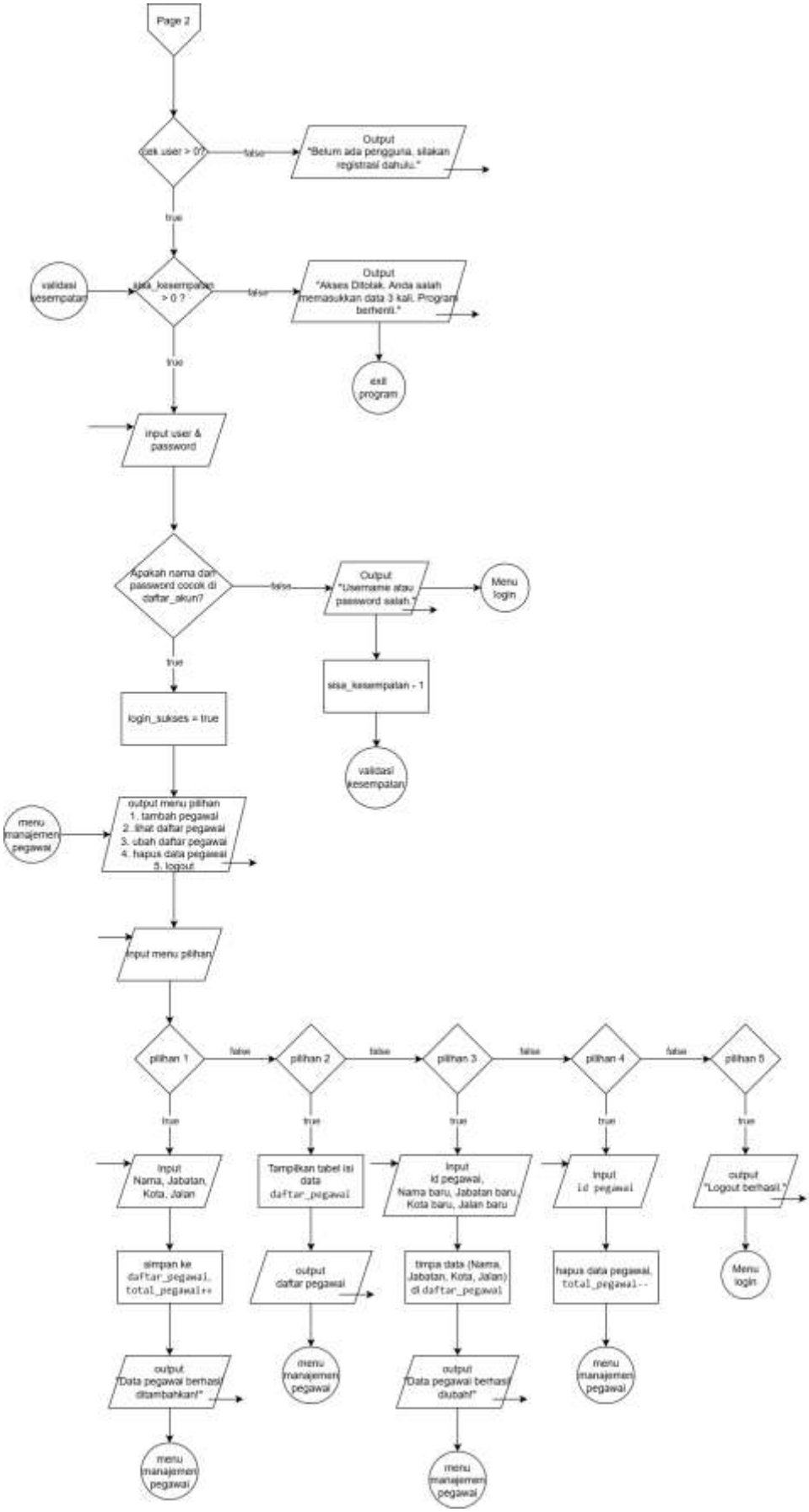
SAMARINDA

2025

1. Flowchart



Gambar 1.1 Flowchart Page 1.



Gambar 1.2 Flowchart Page 2.

Penjelasan Alur Flowchart (Algoritma Program Manajemen Pegawai)

Gambar flowchart di atas menjelaskan alur kerja dari program Manajemen Pegawai Kebab Cendana. Program ini terbagi menjadi dua bagian utama, yaitu subsistem autentikasi (Register & Login) dan subsistem menu utama (CRUD).

Pertama, program menginisialisasi array dan variabel yang dibutuhkan. Kemudian program menampilkan menu autentikasi. Jika pengguna memilih opsi Register, program akan meminta input nama dan password untuk disimpan ke dalam array of struct akun. Jika pengguna memilih opsi Login, program mengatur sisa kesempatan sebanyak 3 kali. Selama kesempatan masih ada, pengguna diminta memasukkan kredensial. Jika data tidak cocok dengan isi array akun, kesempatan berkurang satu. Apabila kesempatan habis, program otomatis berhenti.

Jika login berhasil, alur program akan masuk ke menu utama menggunakan sistem perulangan. Di dalam menu ini, terdapat logika percabangan (switch-case) untuk operasi CRUD. Opsi 1 (Create) meminta input data pegawai dan menyimpannya ke array. Opsi 2 (Read) melakukan perulangan untuk mencetak isi array dalam bentuk tabel. Opsi 3 (Update) dan Opsi 4 (Delete) meminta nomor indeks pegawai untuk mengubah atau menggeser data di dalam array. Setiap selesai memproses opsi, program akan kembali ke menu utama hingga pengguna memilih opsi 5 (Logout) untuk kembali ke menu autentikasi awal.

2. Deskripsi Singkat Program

Program "Manajemen Pegawai Kebab Cendana" merupakan aplikasi interaktif berbasis terminal (CLI) yang dikembangkan menggunakan bahasa C++. Tujuan utama program ini adalah mengimplementasikan materi Array of Struct dan Nested Struct untuk mengelola data pekerja secara terstruktur. Program ini dilengkapi fitur sistem multiuser (Register & Login) dan fitur operasi data lengkap (Create, Read, Update, Delete) yang dibungkus dalam sistem perulangan (looping) agar berjalan otomatis dan efisien.

3. Source Code

A. Deklarasi Struct dan Nested Struct

Fitur ini digunakan untuk mendefinisikan tipe data bentukan yang menyimpan informasi akun dan detail pegawai. Terdapat nested struct (struct di dalam struct) pada bagian Alamat untuk memisahkan data kota dan jalan.

```
struct Akun {
    string nama_pengguna;
    string password_pengguna;
};

// nested struct
struct Alamat {
    string kota;
    string jalan;
};

// main struct
struct Pegawai {
    string nama_pegawai;
    string jabatan;
    Alamat lokasi;
};
```

B. Fitur Autentikasi (Sistem Login Multiuser)

Fitur ini digunakan untuk membatasi akses masuk dengan membandingkan input pengguna terhadap data yang tersimpan di dalam array daftar_akun. Program memberikan batas 3 kali percobaan salah sebelum dihentikan.

```
int sisa_kesempatan = 3;
                    bool login_sukses = false;
                    string input_nama;
                    string input_password;

                    cout << "\n--- LOGIN ---\n";
                    while (sisa_kesempatan > 0 && !login_sukses) {
                        cout << "Masukkan Nama     : ";
                        cin >> input_nama;
                        cout << "Masukkan Password : ";
                        cin >> input_password;
```

```
                        for (int i = 0; i < total_akun; i++) {
                            if (input_nama ==
daftar_akun[i].nama_pengguna && input_password ==
daftar_akun[i].password_pengguna) {
                                login_sukses = true;
                            }
                        }

                        if (login_sukses) {
                            cout << "\n[+] Login Berhasil! Selamat
datang, " << input_nama << ".\n";
                        } else {
                            sisa_kesempatan--;
                            if (sisa_kesempatan > 0) {
                                cout << "[!] Nama atau Password salah!
Sisa percobaan: " << sisa_kesempatan << "\n\n";
                            }
                        }
                    }

                    if (!login_sukses) {
                        cout << "\n[X] Anda gagal login 3 kali. Program
berhenti otomatis.\n";
                        return 0;
                    }
```

C. Fitur Tambah Data Pegawai (Create)

Fitur ini digunakan untuk menambahkan data pegawai baru ke dalam array daftar_pegawai menggunakan indeks penanda total_pegawai yang akan terus bertambah.

```
case 1: {
                                cout << "\n--- TAMBAH PEGAWAI ---\n";
                                cout << "Nama Pegawai : ";
                                cin.ignore();
                                getline(cin,
daftar_pegawai[total_pegawai].nama_pegawai);

                                cout << "Jabatan      : ";
                                getline(cin,
daftar_pegawai[total_pegawai].jabatan);

                                cout << "Kota         : ";
```

```
                                getline(cin,
daftar_pegawai[total_pegawai].lokasi.kota);

                                cout << "Jalan        : ";
                                getline(cin,
daftar_pegawai[total_pegawai].lokasi.jalan);

                                total_pegawai++;
                                cout << "[+] Data pegawai berhasil
ditambahkan!\n";
                                break;
                            }
```

D. Fitur Menampilkan Data (Read dengan Format Tabel)

Fitur ini digunakan untuk membaca dan menampilkan seluruh isi array daftar_pegawai menggunakan perulangan for. Terdapat logika pengecekan panjang karakter (length) agar jarak spasi (tabel) tetap rapi dan lurus.

```
case 2: {
                                cout << "\n--- DAFTAR PEGAWAI KEBAB
CENDANA ---\n";
                                if (total_pegawai == 0) {
                                    cout << "Belum ada data pegawai.\n";
                                } else {
                                    cout << "---------------------------
-----------------------------------------------------\n";
                                    cout << "No.\t| Nama
| Jabatan             | Alamat (Kota, Jalan)\n";
                                    cout << "---------------------------
-----------------------------------------------------\n";

                                    for (int i = 0; i < total_pegawai;
i++) {
                                        cout << i + 1 << "\t| ";

                                        string nama =
daftar_pegawai[i].nama_pegawai;
                                        cout << nama;
                                        for (int s = nama.length(); s <
20; s++) { cout << " "; }
                                        cout << "| ";
```

```
                                        string jabatan =
daftar_pegawai[i].jabatan;
                                        cout << jabatan;
                                        for (int s = jabatan.length(); s
< 20; s++) { cout << " "; }
                                        cout << "| ";

                                        cout <<
daftar_pegawai[i].lokasi.kota << ", "
                                             <<
daftar_pegawai[i].lokasi.jalan << "\n";
                                    }
                                    cout << "--------------------------
-----------------------------------------------------\n";
                                }
                                break;
                            }
```

E. Fitur Ubah Data Pegawai (Update)

Fitur ini memungkinkan pengguna mengubah data yang sudah ada berdasarkan nomor indeksnya. Data baru akan menimpa data lama yang tersimpan di memori array.

```
case 3: {
                                cout << "\n--- UBAH DATA PEGAWAI ---\n";
                                if (total_pegawai == 0) {
                                    cout << "Belum ada pegawai untuk
diubah.\n";
                                } else {
                                    cout << "Masukkan nomor pegawai yang
akan diubah: ";
                                    cin >> index_pegawai;

                                    if (index_pegawai > 0 &&
index_pegawai <= total_pegawai) {
                                        cout << "Masukkan nama baru   :
";
                                        cin.ignore();
                                        getline(cin,
daftar_pegawai[index_pegawai - 1].nama_pegawai);

                                        cout << "Masukkan jabatan baru:
";
                                        getline(cin,
```

```
daftar_pegawai[index_pegawai - 1].jabatan);

                                        cout << "Masukkan kota baru   :
";
                                        getline(cin,
daftar_pegawai[index_pegawai - 1].lokasi.kota);

                                        cout << "Masukkan jalan baru  :
";
                                        getline(cin,
daftar_pegawai[index_pegawai - 1].lokasi.jalan);

                                        cout << "[+] Data berhasil
diubah.\n";
                                    } else {
                                        cout << "[!] Nomor pegawai tidak
valid.\n";
                                    }
                                }
                                break;
                            }
```

F. Fitur Hapus Data Pegawai (Delete)

Fitur ini digunakan oleh pengguna untuk menghapus data berdasarkan nomor urut (indeks). Penghapusan dilakukan dengan cara menggeser (shifting) elemen array ke atas untuk menimpa data yang ingin dihapus.

```
case 4: {
                                cout << "\n--- HAPUS DATA PEGAWAI ---
\n";
                                if (total_pegawai == 0) {
                                    cout << "Belum ada pegawai untuk
dihapus.\n";
                                } else {
                                    cout << "Masukkan nomor pegawai yang
akan dihapus: ";
                                    cin >> index_pegawai;

                                    if (index_pegawai > 0 &&
index_pegawai <= total_pegawai) {
                                        for (int i = index_pegawai - 1;
```

```
i < total_pegawai - 1; i++) {
                                        daftar_pegawai[i] =
daftar_pegawai[i + 1];
                                    }
                                    total_pegawai--;
                                    cout << "[+] Pegawai berhasil
dihapus.\n";
                                } else {
                                    cout << "[!] Nomor pegawai tidak
valid.\n";
                                }
                            }
                            break;
                        }
```

4. Hasil Output

A. Output Menu Register dan Login Berhasil

Tampilan ini menunjukkan simulasi saat pengguna pertama kali mendaftarkan akun, kemudian berhasil melakukan login menggunakan akun tersebut.

```
========================================
       SISTEM PEGAWAI KEBAB CENDANA
========================================

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 1

--- REGISTER ---
Masukkan Nama (Tanpa Spasi) : antung
Masukkan Password           : 092
[+] Akun berhasil didaftarkan! Silakan Login.

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama     : antung
Masukkan Password : 092

[+] Login Berhasil! Selamat datang, antung.

========================================
          MENU MANAJEMEN PEGAWAI
========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan:
```

Gambar 4.1 Output Register dan Login Berhasil

B. Output Gagal Login 3 Kali (Akses Ditolak)

Tampilan ini membuktikan bahwa fitur pembatasan akses berjalan dengan baik. Jika pengguna salah menginput data sebanyak 3 kali, program otomatis dihentikan.

```
--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama     : aweawd
Masukkan Password : wda
[!] Nama atau Password salah! Sisa percobaan: 2

Masukkan Nama     : wad
Masukkan Password : dwa
[!] Nama atau Password salah! Sisa percobaan: 1

Masukkan Nama     : wad
Masukkan Password : adw

[X] Anda gagal login 3 kali. Program berhenti otomatis.
```

Gambar 4.2 Output Login Gagal dan Program Berhenti

C. Output Tambah Pegawai (Create)

Tampilan ini menunjukkan ketika pengguna (admin) berhasil masuk ke menu CRUD dan melakukan penambahan data pegawai beserta alamatnya (Nested Struct).

```
========================================
          MENU MANAJEMEN PEGAWAI
========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 1

--- TAMBAH PEGAWAI ---
Nama Pegawai : antung
Jabatan      : ceo
Kota         : samarinda
Jalan        : cendana
[+] Data pegawai berhasil ditambahkan!
```

Gambar 4.3 Output Tambah Data Pegawai

D. Output Tampilkan Data (Read))

Tampilan ini membuktikan bahwa data yang diinputkan sebelumnya berhasil disimpan di dalam array of struct dan ditampilkan dengan format tabel yang rapi.

```
========================================
          MENU MANAJEMEN PEGAWAI
========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 2

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-------------------------------------------------------------------------------
No.     | Nama                | Jabatan             | Alamat (Kota, Jalan)
-------------------------------------------------------------------------------
1       | antung              | ceo                 | samarinda, cendana
-------------------------------------------------------------------------------
```

Gambar 4.4 Output Tabel Daftar Pegawai

E. Output Ubah dan Hapus Data (Update & Delete)

Tampilan ini menunjukkan proses pencarian data berdasarkan nomor indeks. Data pegawai akan diubah jika memilih opsi 3, atau dihapus jika memilih opsi 4.

```
==========================================
          MENU MANAJEMEN PEGAWAI
==========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 3

--- UBAH DATA PEGAWAI ---
Masukkan nomor pegawai yang akan diubah: 1
Masukkan nama baru   : antoeng
Masukkan jabatan baru: ceo muda aselole
Masukkan kota baru   : balikpapan
Masukkan jalan baru  : sudirman
[+] Data berhasil diubah.

==========================================
          MENU MANAJEMEN PEGAWAI
==========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 4

--- HAPUS DATA PEGAWAI ---
Masukkan nomor pegawai yang akan dihapus: 1
[+] Pegawai berhasil dihapus.
```

Gambar 4.5 Output Ubah dan Hapus Data

F. Output Pilihan Tidak Valid dan Keluar

Tampilan ini menunjukkan ketika pengguna menginput angka di luar daftar menu (contoh: angka 6), program akan menampilkan peringatan dan mengulang menu.

```
==========================================
          MENU MANAJEMEN PEGAWAI
==========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 6
[!] Pilihan tidak valid!
```

Gambar 4.6 Output Validasi Menu

G. Output Exit Program

Tampilan ini menunjukkan ketika pengguna memilih angka 4, program akan berhenti.

```
==========================================
          MENU MANAJEMEN PEGAWAI
==========================================
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 5
Logout berhasil. Kembali ke menu autentikasi...

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 3
Terima kasih telah menggunakan aplikasi Manajemen Pegawai Kebab Cendana!
```

Gambar 4.7 Output Program Selesai

5. Langkah-langkah GIT

5.1 GIT Add

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git add .
```

Perintah git add . berfungsi untuk menambahkan semua file yang baru dibuat atau file yang mengalami modifikasi ke dalam *staging area*.

5.2 GIT Commit

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git commit -m "PT-1"
[main f5782f9] PT-1
 2 files changed, 105 insertions(+)
 create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.cpp
 create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.exe
```

Perintah git commit berfungsi untuk merekam dan menyimpan secara permanen perubahan file yang sudah berada di *staging area* ke dalam riwayat repositori lokal.

5.3 GIT Push

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 448.22 KiB | 6.90 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Perintah git push berfungsi untuk mengunggah semua *commit* atau perubahan yang ada di repositori lokal (komputer) menuju ke repositori *remote* yang ada di Github.